Java Problem Solving

LeetCode #3 — Longest Substring Without Repeating Characters

Problem Statement

Given a string **s**, find the length of the **longest substring** that contains no repeating characters. A substring is a contiguous sequence of characters within the string.

Examples

`Input: s = "abcabcbb"` → `Output: 3 (substring: "abc")`

`Input: s = "bbbbb"` → `Output: 1 (substring: "b")`

`Input: s = "pwwkew"` → `Output: 3 (substring: "wke")`

Approach — Sliding Window + HashMap

We maintain a **sliding window** defined by two pointers, *left* and *right*, that expand over the string. A **HashMap** records the most recent index of each character. When a duplicate is found inside the current window, the *left* pointer jumps just past the previous occurrence, keeping the window valid. The maximum window size seen throughout the scan is returned as the answer.

Java Source Code

```
public class LongestSubstring {

    /**
     * Problem: Longest Substring Without Repeating Characters
     * Given a string s, find the length of the longest
     * substring without repeating characters.
     *
     * Approach: Sliding Window + HashMap
     * Time Complexity : O(n)
     * Space Complexity : O(min(n, m)) where m = charset size
     */

    public static int lengthOfLongestSubstring(String s) {
        java.util.HashMap<Character, Integer> map = new java.util.HashMap<>();
        int maxLen = 0;
        int left = 0;

        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);

            // If char is already in window, move left pointer
            if (map.containsKey(c) && map.get(c) >= left) {
                left = map.get(c) + 1;
            }

            // Update latest index of character
            map.put(c, right);
```

```
            // Update max window size
            maxLen = Math.max(maxLen, right - left + 1);
        }

        return maxLen;
    }

    public static void main(String[] args) {
        String[] testCases = {"abcabcbb", "bbbbb", "pwwkew", "", "abcdef"};
        int[]    expected = {3, 1, 3, 0, 6};

        System.out.println("=== Longest Substring Without Repeating Characters
===");
        System.out.printf("%-15s %-10s %-10s %-8s%n",
                          "Input", "Expected", "Got", "Pass?");
        System.out.println("-".repeat(50));

        for (int i = 0; i < testCases.length; i++) {
            int result = lengthOfLongestSubstring(testCases[i]);
            boolean pass = result == expected[i];
            System.out.printf("%-15s %-10d %-10d %-8s%n",
                """ + testCases[i] + """,
                expected[i], result,
                pass ? "PASS" : "FAIL");
        }
    }
}
```

Complexity Analysis

Metric	Value	Reason
Time Complexity	O(n)	Single pass through string
Space Complexity	O(min(n,m))	HashMap stores at most charset size entries
Best Case	O(n)	All unique chars — no left pointer moves
Worst Case	O(n)	All duplicate chars — left pointer moves every step

Key Takeaways

• The sliding window pattern avoids nested loops, reducing brute-force $O(n^2)$ or $O(n^3)$ solutions down to O(n).
• Storing the *index* in the HashMap (not just presence) lets us jump the left pointer in O(1) rather than crawling it forward.
• The guard `map.get(c) >= left` ensures we only react to duplicates *inside* the current window, ignoring stale entries.

Difficulty: Medium | Topic: Sliding Window, HashMap | Language: Java 17+ | LeetCode #3